

2.02-hpi-partialresponses-save

September 2, 2024

```
[5]: %load_ext autoreload
      %autoreload 2

      import numpy as np
      import pandas as pd
      from datetime import datetime

      from prism.config import PROCESSED_DATA_DIR, MODELS_DIR
      from prism.load_models import load_mlp
      from prism.pr_save_test import save_partial_responses
      from prism.legacy import normalise
      from prism.obsolete.partial_responses_old import partialResponses
      from prism.obsolete.prplots_old import prPlots
```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

```
[19]: %reload_ext autoreload
```

0.1 Load and preprocess data

```
[7]: data_train = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_train.
      ↪csv'))
      data_test = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_test.
      ↪csv'))
      data_val = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_val.csv'))

      data_train_test = pd.concat([data_train, data_test]) # used for axis annotation,
      ↪in some plotting functions

      # drop id column
      data_train.drop('trr_id_code', axis=1, inplace=True)
      data_test.drop('trr_id_code', axis=1, inplace=True)
      data_val.drop('trr_id_code', axis=1, inplace=True)

      target_col = 'oneyearmort'

      x_train0 = data_train.drop(target_col, axis=1)
```

```

y_train = data_train[target_col]

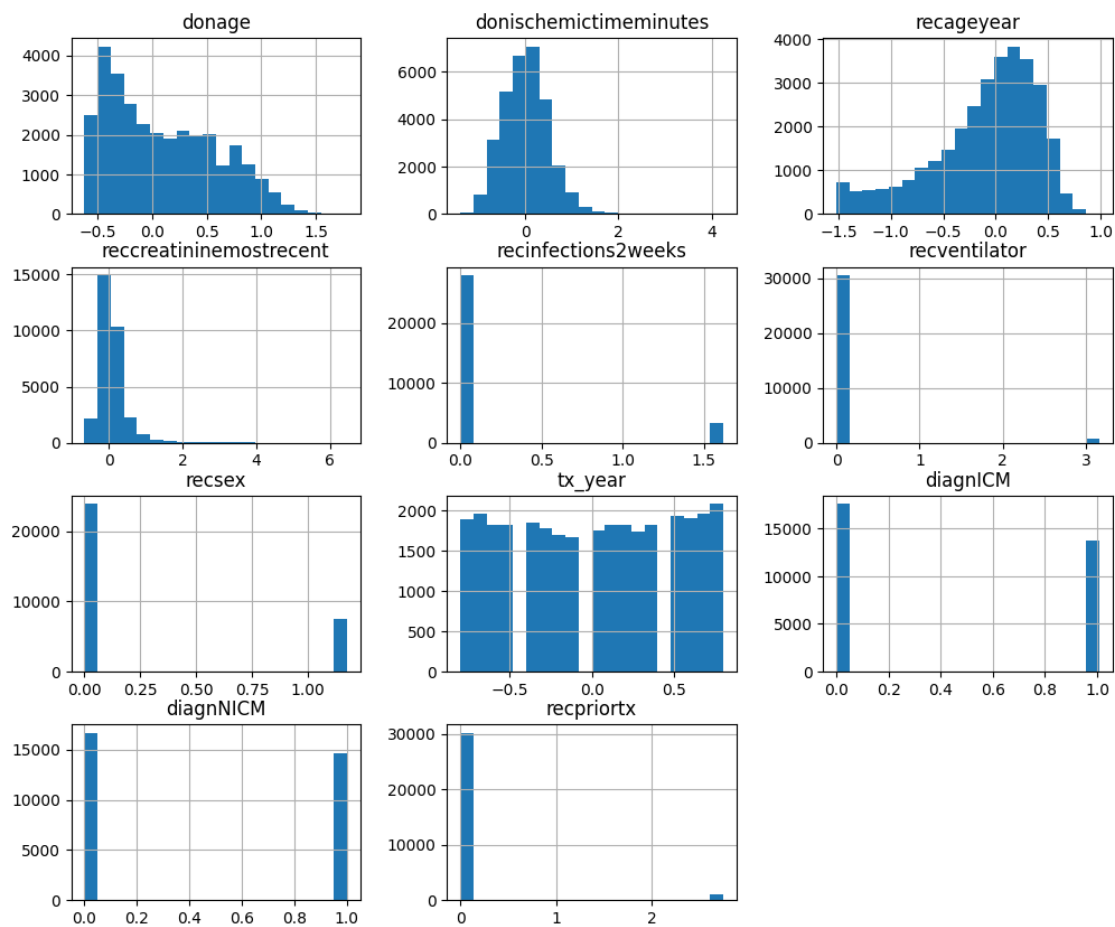
x_test0 = data_test.drop(target_col,axis=1)
y_test = data_test[target_col]

x_val0 = data_val.drop(target_col,axis=1)
y_val = data_val[target_col]

[x_train,x_test] = normalise(x_train0,x_test0)
x_val = normalise(x_val0)

```

```
[8]: x_train.hist(bins=20,figsize=(12,10));
```



0.2 Load MLP

```
[10]: filename_mlp = 'mlp_model_20240705_135534'
try:
    model, params, metrics = load_mlp(filename_mlp, MODELS_DIR)
    print("Model loaded successfully")
    print("Derived Model Structure:")
    print("\nModel Parameters:")
    print(params)
    print("\nModel Metrics:")
    print(metrics)
except Exception as e:
    print(f"Error loading model: {e}")
    raise

# Extract method and device from params, with defaults
method = params.get('method', 'dirac')
device = params.get('device', 'cpu')
```

Model loaded successfully

Derived Model Structure:

Model Parameters:

```
{'n_hidden': 10, 'weight_decay': 1e-05, 'lr': 0.001, 'patience': 50,
'tolerance': 0.0001, 'batch_size': 1024, 'device': 'cpu', 'seed': 257}
```

Model Metrics:

```
{'prevalence': 0.123, 'sensitivity': 0.241, 'specificity': 0.864, 'accuracy':
0.797, 'ppv': 0.175, 'auc score': 0.601, 'auc lower ci': '0.577', 'auc upper
ci': '0.626'}
```

0.3 Calculate and save partial responses (existing benchmark implementation)

```
[25]: timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
pr_train_benchmark, pr_test_benchmark, bivariate_inputs =
    ↪ save_partial_responses(x_train, x_test, model, method=method, device=device,
    ↪ filename="partial_responses_data_{timestamp}.pkl")

print(f"Partial responses calculated using method '{method}' on device
    ↪ '{device}' and saved.")
```

Univariate Responses:

Column 0:

Column 1:

Column 2:

Column 3:

Column 4:

Column 5:

Column 6:
Column 7:
Column 8:
Column 9:
Column 10:
Bivariate Responses:
Columns 0 & 1:
Columns 0 & 2:
Columns 0 & 3:
Columns 0 & 4:
Columns 0 & 5:
Columns 0 & 6:
Columns 0 & 7:
Columns 0 & 8:
Columns 0 & 9:
Columns 0 & 10:
Columns 1 & 2:
Columns 1 & 3:
Columns 1 & 4:
Columns 1 & 5:
Columns 1 & 6:
Columns 1 & 7:
Columns 1 & 8:
Columns 1 & 9:
Columns 1 & 10:
Columns 2 & 3:
Columns 2 & 4:
Columns 2 & 5:
Columns 2 & 6:
Columns 2 & 7:
Columns 2 & 8:
Columns 2 & 9:
Columns 2 & 10:
Columns 3 & 4:
Columns 3 & 5:
Columns 3 & 6:
Columns 3 & 7:
Columns 3 & 8:
Columns 3 & 9:
Columns 3 & 10:
Columns 4 & 5:
Columns 4 & 6:
Columns 4 & 7:
Columns 4 & 8:
Columns 4 & 9:
Columns 4 & 10:
Columns 5 & 6:
Columns 5 & 7:

```

Columns 5 & 8:
Columns 5 & 9:
Columns 5 & 10:
Columns 6 & 7:
Columns 6 & 8:
Columns 6 & 9:
Columns 6 & 10:
Columns 7 & 8:
Columns 7 & 9:
Columns 7 & 10:
Columns 8 & 9:
Columns 8 & 10:
Columns 9 & 10:
Residual
Data saved to partial_responses_data.pkl
Partial responses calculated using method 'dirac' on device 'cpu' and saved.

```

```
[34]: pr_train_benchmark.shape
```

```
[34]: (31315, 66)
```

```
[44]: x_train.columns[2]
```

```
[44]: 'recageyear'
```

```
[45]: import matplotlib.pyplot as plt

# Univariate responses only
n = pr_train_benchmark.shape[0]
m = 11

# Calculate number of rows needed for the subplots
num_subplot_cols = 3
num_subplot_rows = (m + num_subplot_cols - 1) // num_subplot_cols

# Create a figure with subplots arranged in 3 columns
fig, axes = plt.subplots(num_subplot_rows, num_subplot_cols, figsize=(12, 4 *
    ↳num_subplot_rows))
fig.suptitle(f'Histograms of Benchmark Training Partial Responses')

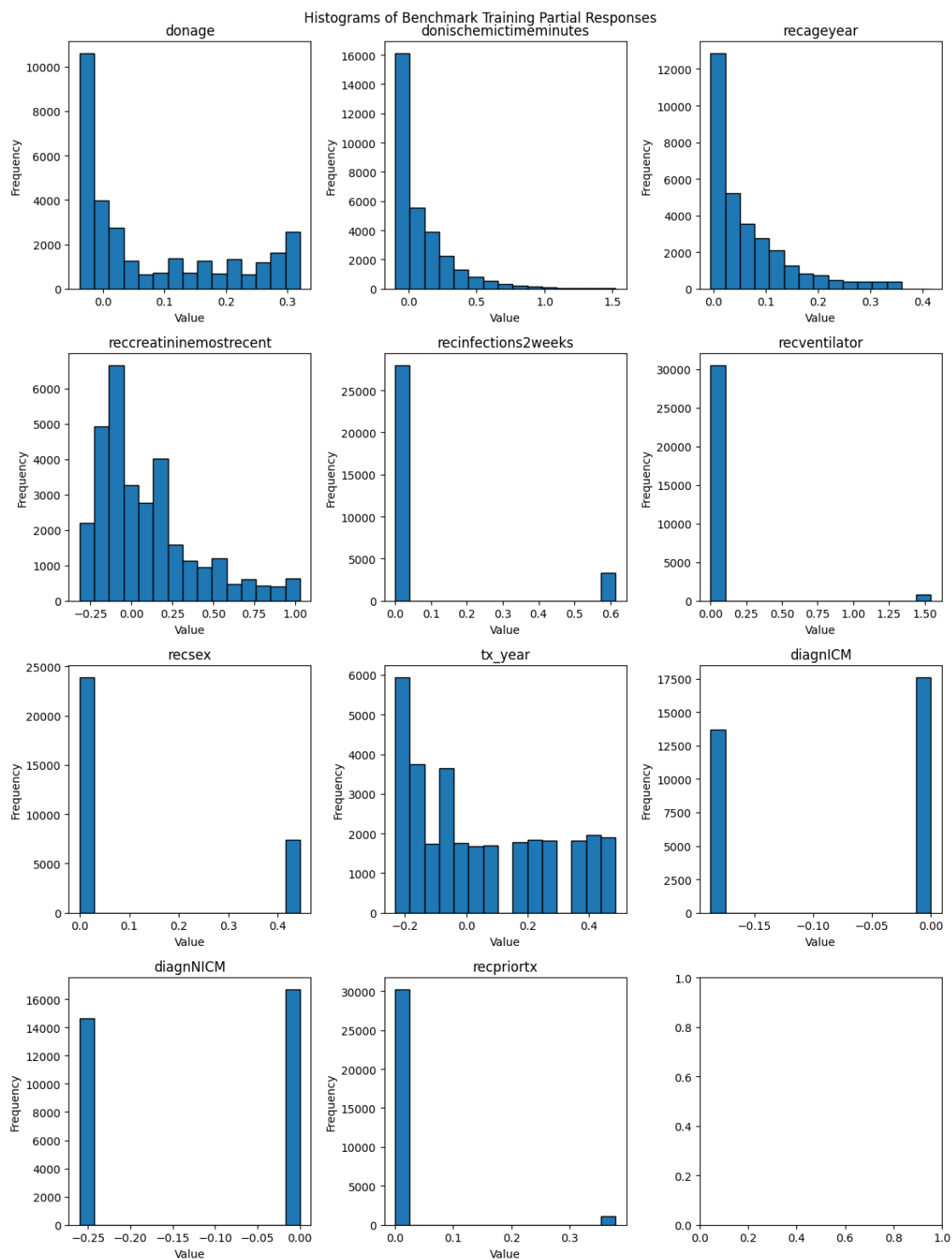
# Plot histogram for each column
for i in range(m):
    row = i // num_subplot_cols
    col = i % num_subplot_cols
    ax = axes[row, col]
    ax.hist(pr_train_benchmark[:, i], bins=15, edgecolor='black')
    ax.set_title(x_train.columns[i])
    ax.set_xlabel('Value')
```

```
ax.set_ylabel('Frequency')
```

```
# Adjust layout and show the plot
```

```
plt.tight_layout()
```

```
plt.show()
```



0.4 Plot partial responses

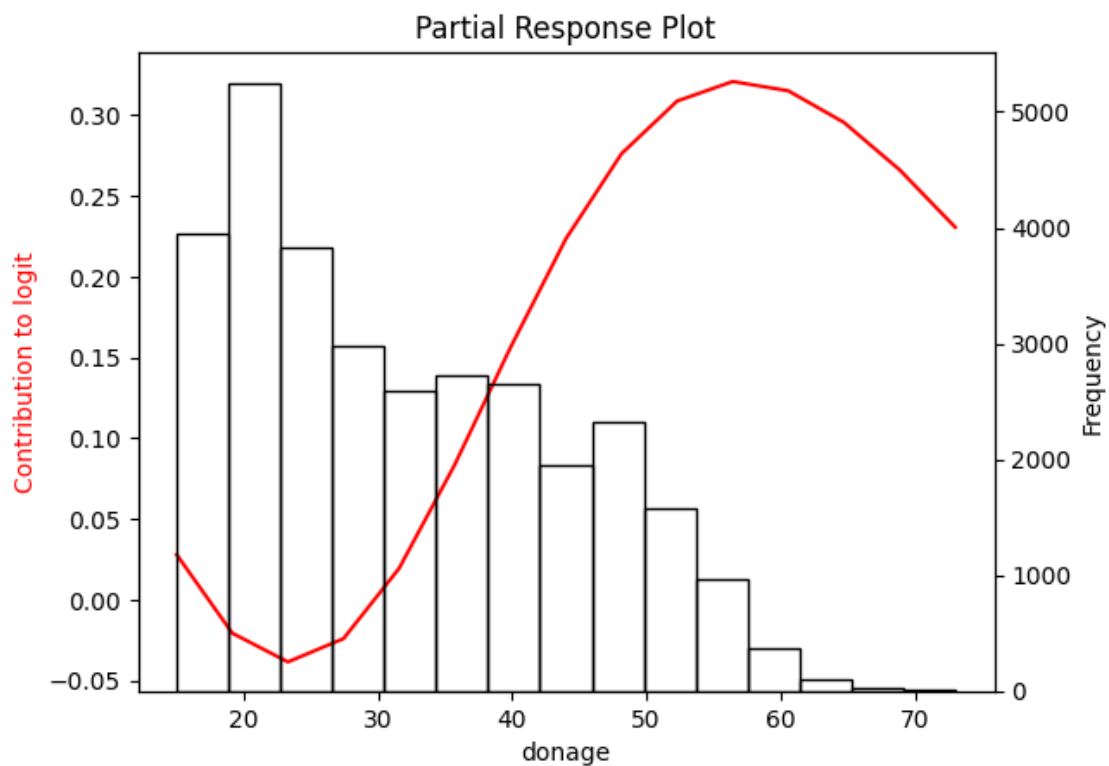
```
[26]: pr_test_benchmark.shape, x_test.shape
```

```
[26]: ((6120, 66), (6120, 11))
```

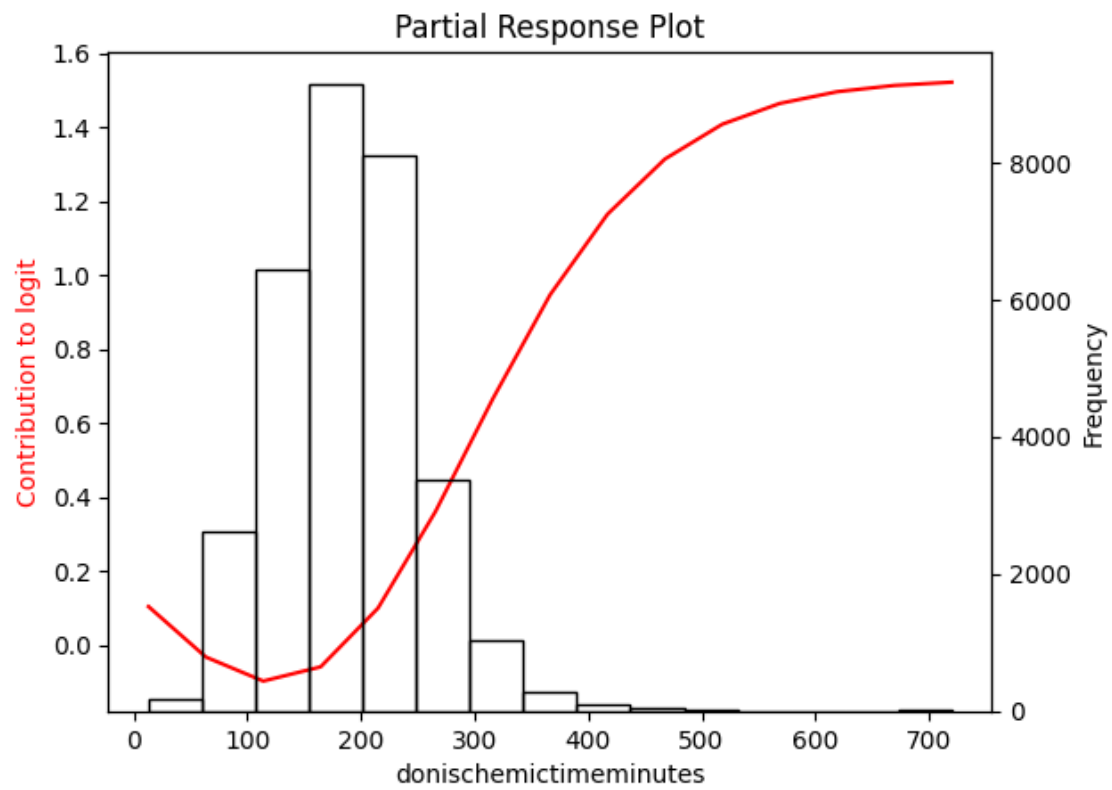
```
[27]: betas_univariate = np.zeros([pr_train_benchmark.shape[1],1])  
betas_univariate[:x_train.shape[1],0] = 1
```

```
[28]: prPlots(betas_univariate, 0, x_train0, x_train, data_train_test, model,   
↪ bivariate_inputs, n_steps = 15, sd_scale=2, method=method, device=device)
```

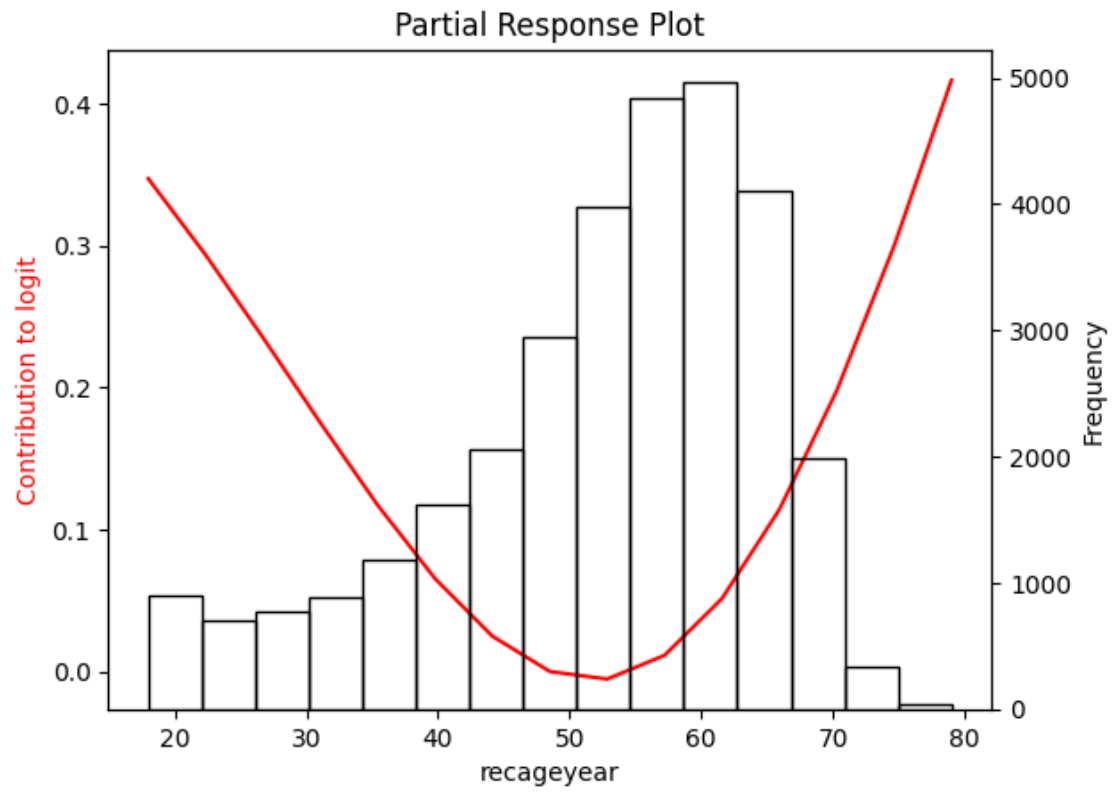
0 - univ



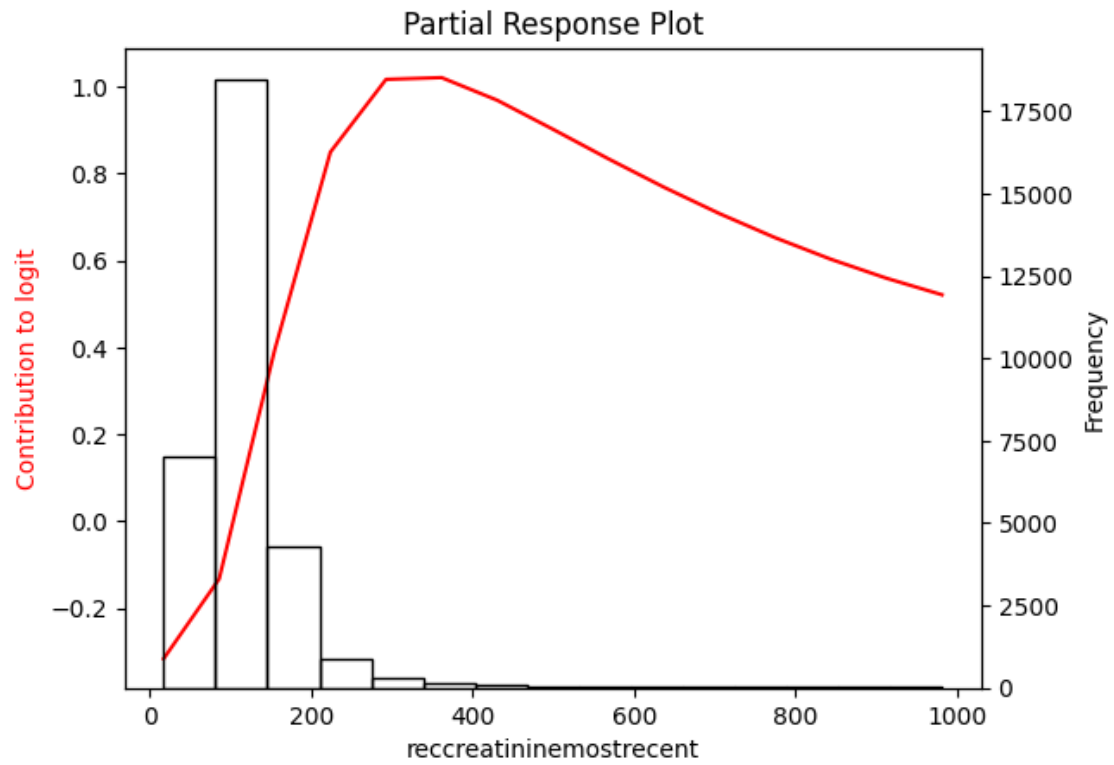
1 - univ



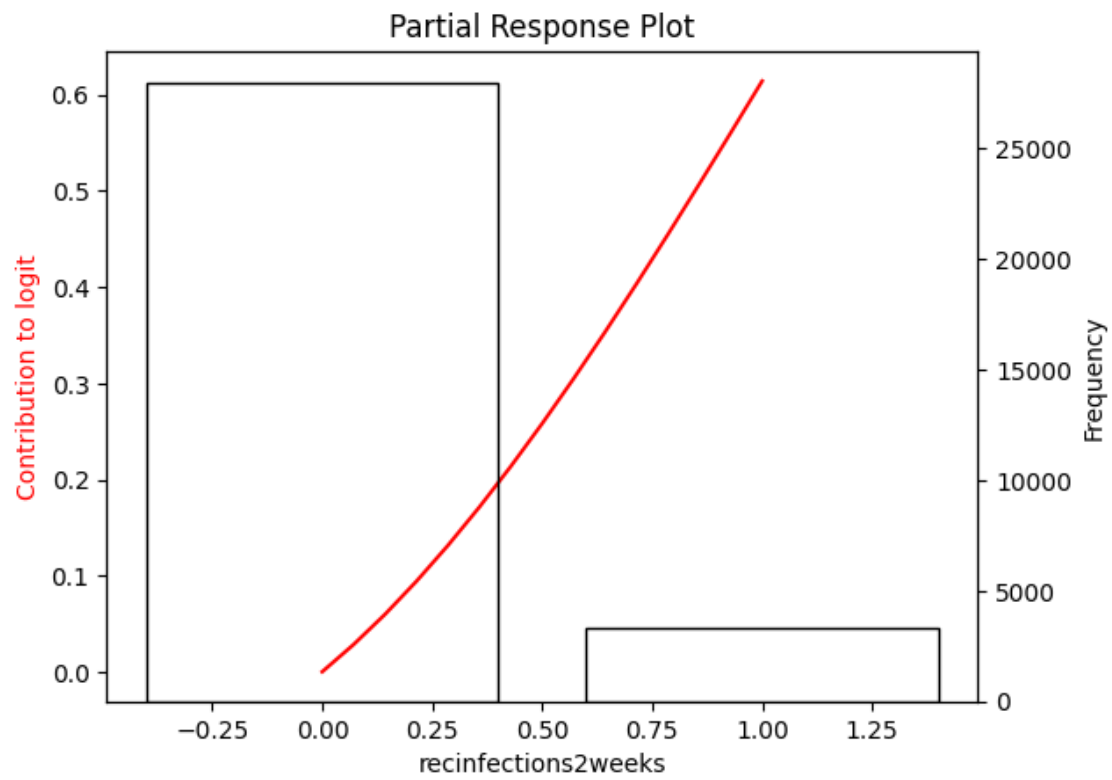
2 - univ



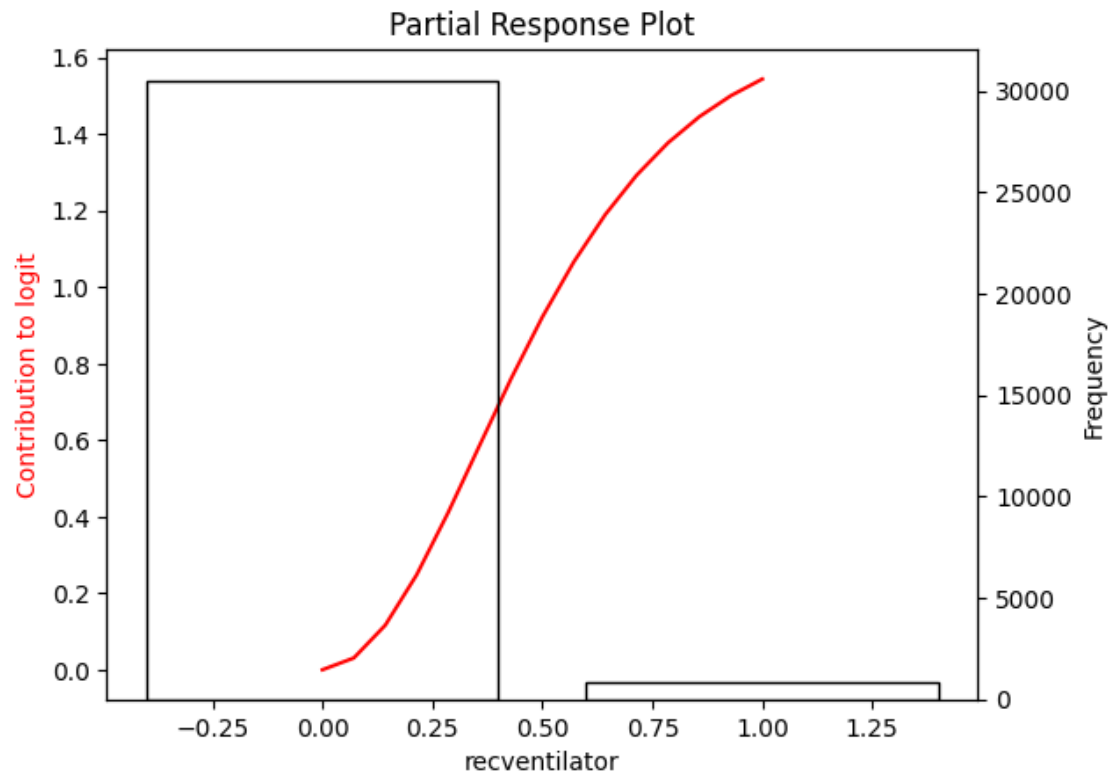
3 - univ



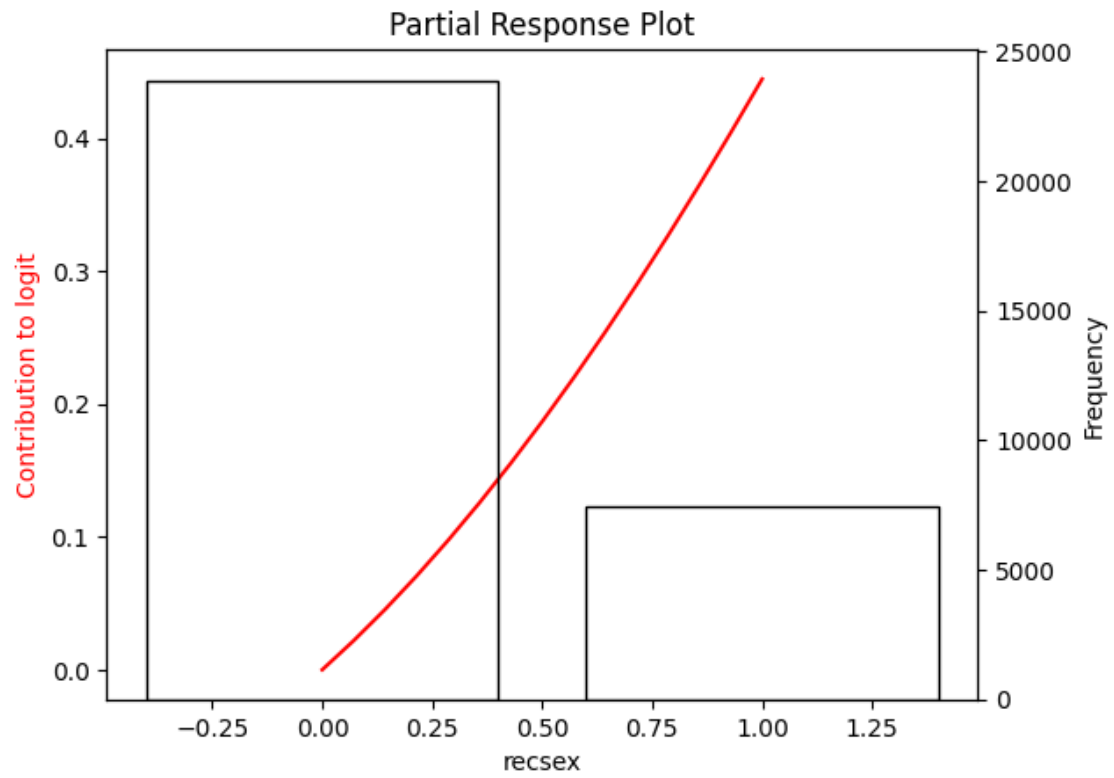
4 - univ



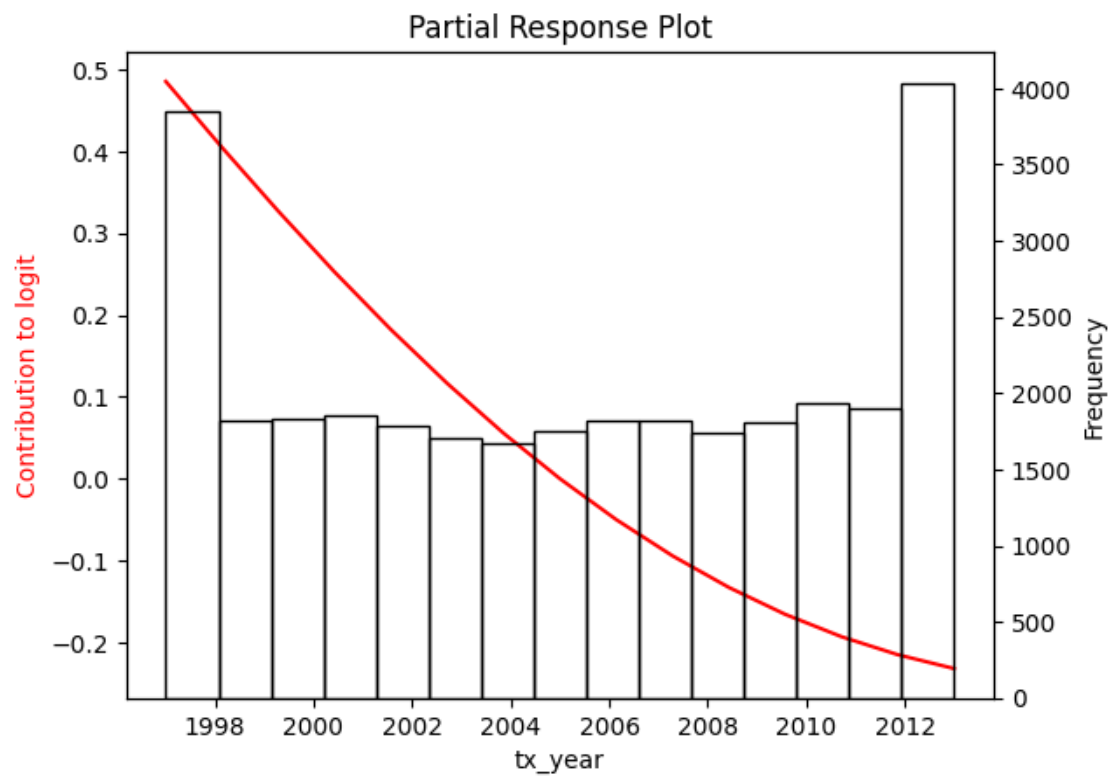
5 - univ



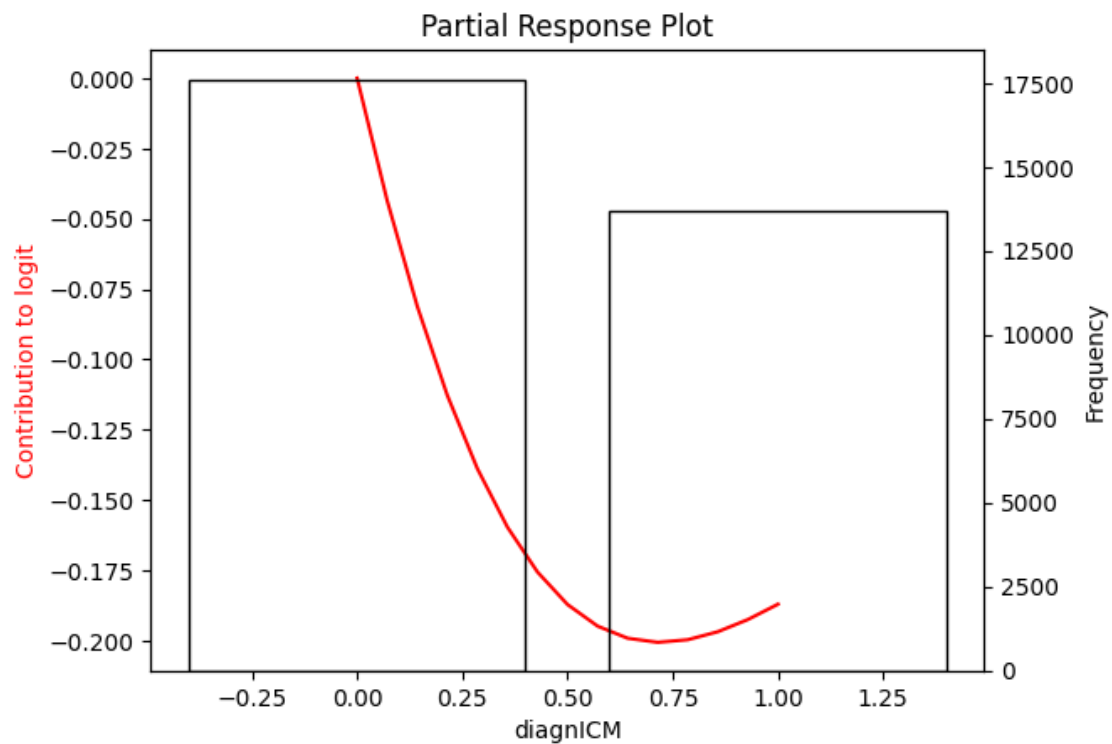
6 - univ



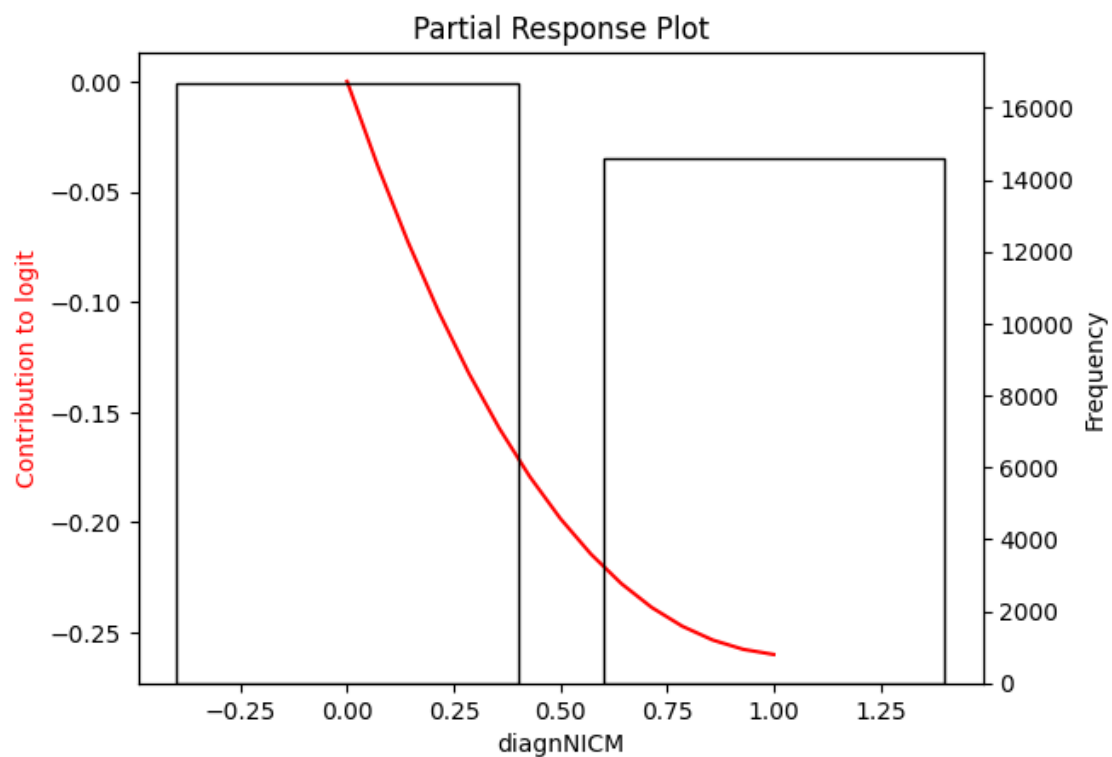
7 - univ



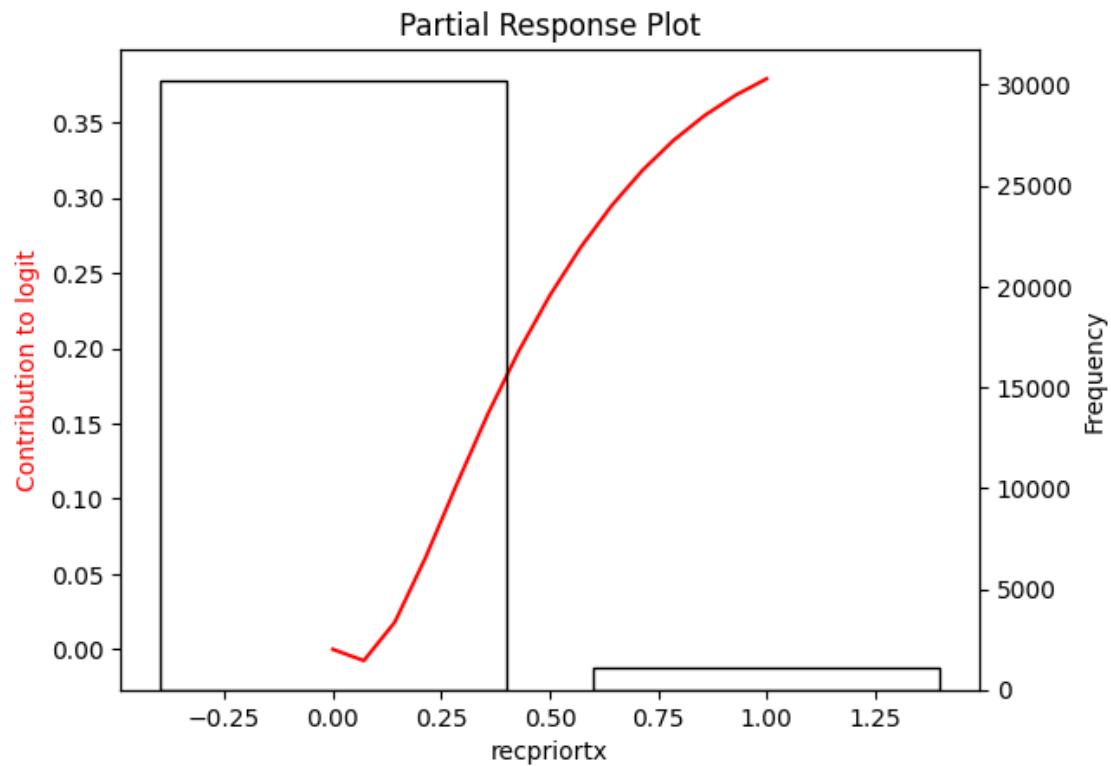
8 - univ



9 - univ



10 - univ



0.5 Test the saved partial responses to verify test function

```
[24]: test_result = test_refactored_partial_responses(partialResponses,
    ↪filename="partial_responses_data.pkl")

if test_result:
    print("Test passed.")
else:
    print("Test failed.")
```

Univariate Responses:

Column 0:

Column 1:

Column 2:

Column 3:

Column 4:

Column 5:

Column 6:

Column 7:
Column 8:
Column 9:
Column 10:
Bivariate Responses:
Columns 0 & 1:
Columns 0 & 2:
Columns 0 & 3:
Columns 0 & 4:
Columns 0 & 5:
Columns 0 & 6:
Columns 0 & 7:
Columns 0 & 8:
Columns 0 & 9:
Columns 0 & 10:
Columns 1 & 2:
Columns 1 & 3:
Columns 1 & 4:
Columns 1 & 5:
Columns 1 & 6:
Columns 1 & 7:
Columns 1 & 8:
Columns 1 & 9:
Columns 1 & 10:
Columns 2 & 3:
Columns 2 & 4:
Columns 2 & 5:
Columns 2 & 6:
Columns 2 & 7:
Columns 2 & 8:
Columns 2 & 9:
Columns 2 & 10:
Columns 3 & 4:
Columns 3 & 5:
Columns 3 & 6:
Columns 3 & 7:
Columns 3 & 8:
Columns 3 & 9:
Columns 3 & 10:
Columns 4 & 5:
Columns 4 & 6:
Columns 4 & 7:
Columns 4 & 8:
Columns 4 & 9:
Columns 4 & 10:
Columns 5 & 6:
Columns 5 & 7:
Columns 5 & 8:

```

Columns 5 & 9:
Columns 5 & 10:
Columns 6 & 7:
Columns 6 & 8:
Columns 6 & 9:
Columns 6 & 10:
Columns 7 & 8:
Columns 7 & 9:
Columns 7 & 10:
Columns 8 & 9:
Columns 8 & 10:
Columns 9 & 10:
Residual
Refactored function produces similar results to the original implementation.
Test passed.

```

1 Save Lebesgue CPU 2000

```

[23]: import psutil

# Get memory information
mem = psutil.virtual_memory()

# Process-specific memory info
process = psutil.Process()
process_mem = process.memory_info()

print(f"Total memory: {mem.total / (1024 * 1024):.2f} MB")
print(f"Available memory: {mem.available / (1024 * 1024):.2f} MB")
print(f"Used memory: {mem.used / (1024 * 1024):.2f} MB")
print(f"Memory usage percentage: {mem.percent:.2f}%")

# Check for Linux-specific attributes
if hasattr(mem, 'cached'):
    print(f"Cached memory: {mem.cached / (1024 * 1024):.2f} MB")
if hasattr(mem, 'buffers'):
    print(f"Buffered memory: {mem.buffers / (1024 * 1024):.2f} MB")

print("\nCurrent process memory usage:")
print(f"RSS (Resident Set Size): {process_mem.rss / (1024 * 1024):.2f} MB")
print(f"VMS (Virtual Memory Size): {process_mem.vms / (1024 * 1024):.2f} MB")

# Check for platform-specific attributes
if hasattr(process_mem, 'shared'):
    print(f"Shared memory: {process_mem.shared / (1024 * 1024):.2f} MB")

```

Total memory: 32045.02 MB

Available memory: 13078.40 MB
Used memory: 18966.62 MB
Memory usage percentage: 59.20%

Current process memory usage:
RSS (Resident Set Size): 581.86 MB
VMS (Virtual Memory Size): 1699.06 MB

```
[25]: timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
device = 'cpu'
method = 'lebesgue'
pr_train_benchmark, pr_test_benchmark, bivariate_inputs =
    ↪save_partial_responses(x_train.iloc[:2000], x_test.iloc[:2000], model,
    ↪method=method, device=device, filename=f"pr_{device}_{method}_{timestamp}.
    ↪pkl")

print(f"Partial responses calculated using method '{method}' on device
    ↪'{device}' and saved.")
```

Univariate Responses:

```
Train pred: iterating over all features at x sample 0
Train pred: iterating over all features at x sample 100
Train pred: iterating over all features at x sample 200
Train pred: iterating over all features at x sample 300
Train pred: iterating over all features at x sample 400
Train pred: iterating over all features at x sample 500
Train pred: iterating over all features at x sample 600
Train pred: iterating over all features at x sample 700
Train pred: iterating over all features at x sample 800
Train pred: iterating over all features at x sample 900
Train pred: iterating over all features at x sample 1000
Train pred: iterating over all features at x sample 1100
Train pred: iterating over all features at x sample 1200
Train pred: iterating over all features at x sample 1300
Train pred: iterating over all features at x sample 1400
Train pred: iterating over all features at x sample 1500
Train pred: iterating over all features at x sample 1600
Train pred: iterating over all features at x sample 1700
Train pred: iterating over all features at x sample 1800
Train pred: iterating over all features at x sample 1900
Test pred: iterating over all features at x sample 0
Test pred: iterating over all features at x sample 100
Test pred: iterating over all features at x sample 200
Test pred: iterating over all features at x sample 300
Test pred: iterating over all features at x sample 400
Test pred: iterating over all features at x sample 500
Test pred: iterating over all features at x sample 600
Test pred: iterating over all features at x sample 700
```

```
Test pred: iterating over all features at x sample 800
Test pred: iterating over all features at x sample 900
Test pred: iterating over all features at x sample 1000
Test pred: iterating over all features at x sample 1100
Test pred: iterating over all features at x sample 1200
Test pred: iterating over all features at x sample 1300
Test pred: iterating over all features at x sample 1400
Test pred: iterating over all features at x sample 1500
Test pred: iterating over all features at x sample 1600
Test pred: iterating over all features at x sample 1700
Test pred: iterating over all features at x sample 1800
Test pred: iterating over all features at x sample 1900
Bivariate Responses:
Residual
Data saved to pr_cpu_lebesgue_20240816_170150.pkl
Partial responses calculated using method 'lebesgue' on device 'cpu' and saved.
```

[]: